

ASP.NET Core CON C#

10. Bases de datos

Creamos la base de datos de la misma forma que para proyectos de escritorio: “abrir conexión de datos”. Una vez que la tengamos, definimos la cadena de conexión

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=C:\\Users"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Para llevar a cabo la conexión y las operaciones tenemos varias librerías: Dapper, Entity Framework core,,,

Entity Framework Core (EF Core)

Es una librería para acceder a bases de datos. En términos técnicos es lo que se conoce como un ORM (Object Relation Mapper) es una herramienta que nos permite mapear, o lo que es lo mismo, convertir los objetos de tu aplicación a un formato adecuado para ser almacenados en cualquier base de datos. Nos permite representar en objetos las tablas de una bbdd, trabajaremos sin “escribir” el SQL. Es multiplataforma (Windows,Linux,mac)

En noviembre de cada año sale una versión nueva. En noviembre de 2022 se publicó la versión 7. Las versiones pares son LTF (long time support, 3 años. A los otros 2 años).

Se utiliza cuando tenemos un fabricante que nos permite poder realizar consultas Linq hacia un origen de datos.

Language-Integrated Query (LINQ) es el nombre de un conjunto de tecnologías basadas en la integración de capacidades de consulta directamente en el lenguaje C#.

Dapper vs EntityFramework

Dapper es un micro-ORM, esto significa que es menos pesado, pero por el contrario hay que escribir todas las sentencias SQL.

En las sucesivas versiones de Entity Framework se mejora constantemente tanto la velocidad como la memoria utilizada

LINQ en C#

La sintaxis es parecida a la existente en SQL, aunque no estemos accediendo a una base de datos

```
var lista = from c in coleccion where c.propiedad1 == true select c;
```

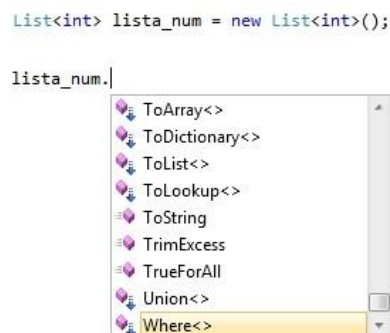
Accedemos a una colección y filtramos todos los elementos que cumplan que la propiedad sea verdadera. Del resultado de esta consulta, obtenemos un listado de elementos con ToList(), el número de ellos con un count(), extraer los datos a un array, etc...

```
lista.ToList()
```

Podemos utilizar expresiones lambda para llamar directamente a funciones where, join, select

```
var lista_resultado = lista.where(c => c.id > 5).OrderBy(c => c);
```

En solo una línea de código, a partir de una lista, hemos filtrado los objetos de la misma que cumplen una condición y hemos ordenado los resultados.



```
List<Ejemplo> l = new List<Ejemplo>() { new Ejemplo(3, true),  
                                         new Ejemplo(2, false) };
```

```
var uno= l.Where(c => c.Filtro == true);  
foreach (var u in uno)  
    listBox1.Items.Add(u.X);
```

¿Qué es LINQ para SQL?

LINQ para SQL es una implementación de ORM (Object Relational Mapping) que viene con .NET Framework y nos permite modelar bases de datos relacionales con clases de .NET. Podemos consultar, actualizar, añadir, borrar datos...

Empezamos con EF

Es necesario tener actualizado el VS porque si no las versiones que se instalan no serán soportadas por nuestro framework. Para actualizar, ejecutamos en nuestra máquina el Visual Studio Installer

Al empezar un proyecto que requiere de una bbdd, debemos tener en mente si seguimos un proceso de “Code first” o de “Database first”.

Code first es el proceso en el que el desarrollador escribe las clases que van a ser utilizadas como entidades y entity framework se encarga de crear las tablas en base a esas clases.

Database First es el proceso en el que la base de datos se crea de forma manual (a través de SQL) o ya existe anteriormente y es Entity Framework el que se encarga de generar las entidades basándose en dicha base de datos.

Si no tenemos bbdd:

1. Implementamos las clases que representan las bbdd
2. Configuramos las relaciones
3. Generamos la bbdd desde nuestro código

Si ya tenemos la bdd:

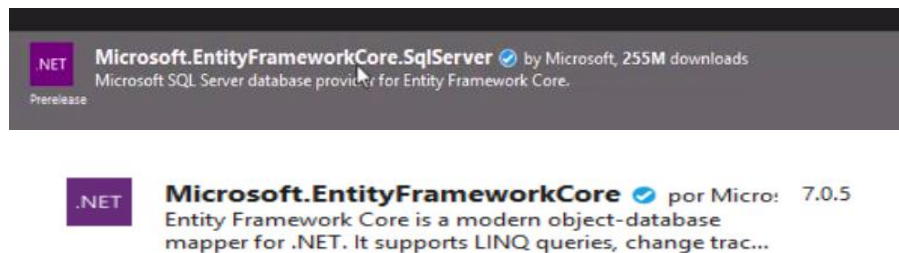
1. Nos conectamos a la bbdd e implementamos los modelos correspondientes a las tablas. Mediante decoraciones mapearemos esas clases dentro de Visual Studio.

Instalación

Buscamos en Google ef core cli, en la página de Microsoft vemos el comando para instalar

```
C:\Users\admin>dotnet tool install --global dotnet-ef  
  
Esto es .NET 8.0.  
-----  
Versión del SDK: 8.0.101
```

Una vez instalada la herramienta, en mi aplicación debo instalar algunos paquetes NuGet:



Para utilizar EF con sqlserver. Hay paquetes para otros motores de bbdd



La estructura es la siguiente:

1. Modelos de la base de datos mapeados
2. Objeto Context que contiene las colecciones DbSet de los modelos.
3. Servicios para las consultas Linq sobre Context
4. Añadir configuraciones en Program.cs
5. Inyección de dependencias para resolver el Context

Mediante Mapping de modelos, podemos indicar el enlace del Model con la tabla. Se realiza mediante decoraciones:

- [Table]: Indica el nombre de la tabla
- [Column]: Nombre de la columna
- [Key]: Primary Key de la tabla. Solamente es obligatorio si realizamos consultas de acción
- [DatabaseGenerated]: Si nuestras tablas tienen autoincremento, con ello se lo indicamos.

Clase DbContext

DbContext es una clase abstracta que se puede heredar para crear una clase concreta que represente el contexto de la base de datos de nuestra aplicación. Esta clase concreta se utiliza para configurar la conexión a la base de datos, definir las entidades que se almacenarán en la base de datos y aplicar cambios en la base de datos. Es la clase principal de trabajo que se utiliza para realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) en la base de datos.

Crearemos por tanto una clase nuestra que herede de ésta

Tendrá un constructor que debe recibir, mediante inyección, las opciones para la base de datos.

Contendrá colecciones de tipo DbSet que deben ser propiedades y nos permitirán acceder a los datos.

DbSet es una clase en Entity Framework Core que representa un conjunto de entidades en la base de datos. Cada propiedad DbSet en una clase DbContext representa una tabla en la base de datos.

Configurar la Conexión

Lo primero es configurar la cadena de conexión. Añadimos la opción ConnectionStrings en el archivo appsettings(developer). Esta cadena se obtiene en el archivo Program.cs

```
using EjemploEF;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
{ options.UseSqlServer("name=DefaultConnection"); }
);
```

Una vez configurada la conexión, podemos empezar a trabajar:

1. Las tablas las creamos desde nuestra aplicación (no existen)
2. Las tablas ya existen

En ambos casos sí tenemos creado el archivo mdf

Empezamos trabajando sobre una bbdd con tablas ya creadas.

Pasos en la Implementación

1. Implementar los modelos correspondientes a las tablas utilizando los decoradores de la clase `System.ComponentModel.DataAnnotations` para reflejar en las propiedades de mi modelo las características de los campos de mi tabla.
2. Implementar mi clase `Context` heredando de `DbContext`
3. Implementar mis servicios, utilizarán la clase anterior para acceder a la bbdd
4. Resolver las dependencias en `Program.cs`
5. Implementar Controladores y Vistas

Acceso a los datos

1. Consultas de Selección: Aquellas que no implican modificar datos(select)

- Todos los registros `toList()`

```
Contexto.DBsetClase.toList()
```

Aunque podemos usar otras opciones

```
from datos in "Contexto"  
select datos;
```

Este formato es LINQ (Véanse más expresiones en la documentación de Microsoft)

- Un solo registro (entidad)

```
Contexto.DbSetClase.Single(e=>{e.Propiedad==....})  
También podemos usar los métodos First
```

- Obtener sólo algunas columnas

```
Contexto.DBSetClase. .Select(e => $"{e.Propiedad} {e.Propiedad}")  
    .ToList();
```

- Filtrado

```
Context.DBSetClase.Where( b => condición)  
    .ToList();
```

- Búsqueda por Primary Key

```
Context.DBSetClase.Find(b=>condición)
```

Este método se usa para buscar por la clave primaria por lo que podemos pasar por parámetro al método directamente ese valor

```
Context.DbSetClase.Find(valor)
```

2. Consultas de Acción EF (las hacemos asíncronas)

- Insertar

```
Context.ColeccionDbSet.Add(model);  
Context.SaveChanges();
```

- Eliminar

Buscamos el modelo

```
Context.ColeccionDbSet.Remove(model);  
Context.SaveChanges();
```

- Modificar

Buscamos el modelo

```
model.Propiedad1 = "Valor1";  
model.Propiedad2 = "Valor2";  
Context.SaveChanges();
```

Todo puede ser transaccional, es decir, podemos realizar múltiples consultas de acción y, cuando deseemos, llevarlas todas a la base de datos. Para guardar los cambios en la base de datos se utiliza el método `SaveChanges()` del objeto `Context`

Ejemplo: “Ciclos y alumnos”

1. Implementamos el *modelo* Ciclo añadiendo a cada campo el decorador correspondiente
2. Añadimos una nueva carpeta llamada Data para almacenar nuestra clase Contexto (CicloContexto)
3. Implementamos la clase correspondiente al servicio para esa tabla en la carpeta Repositorio
4. Resolvemos las dependencias en la clase Program
5. Implementamos un controlador con la correspondiente acción y vista

Completamos el ejemplo:

6. Añadimos una opción para crear un nuevo ciclo
7. Añadimos la parte de los alumnos, tendremos que dar los mismos pasos que para la parte de los ciclos

Podemos ejecutar consultas SQL tan complejas como sea necesario

```
var modelo = cuentasConTipoCuenta
    .GroupBy(x => x.TipoCuenta)
    .Select(grupo => new IndiceCuentasViewModel
    {
        TipoCuenta = grupo.Key,
        Cuentas = grupo.AsEnumerable()
    }).ToList();
```

Agrupamos los registros por su campo TipoCuentas, para cada elemento de esa agrupación generamos un objeto con dos propiedades: grupo.key no es más que el valor por el que agrupamos antes y grupo son los registros “agrupados” por lo que los convertimos en una colección IEnumerable.

Podemos también añadir propiedades a los objetos que sean sumatorios

```
3 referencias
public class Tarea {
    0 referencias
    public int Id { get; set; }

    [Required]
    [StringLength(200)]
    0 referencias
    public string Titulo { get; set; }
    1 referencia
    public string Descripcion { get; set; }
    0 referencias
    public int Orden { get; set; }
    0 referencias
    public DateTime FechaCreacion { get; set; }

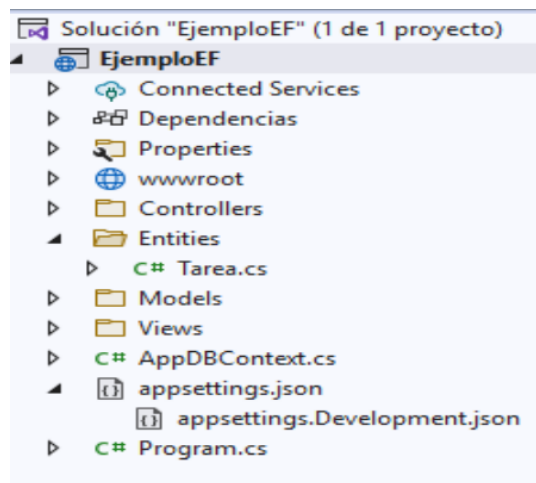
    0 referencias
    public List<Item> Items { get; set; }
}
```

```
1 referencia
public class Item
{
    0 referencias
    public Guid Id { get; set; }
    0 referencias
    public int TareaId { get; set; }
    0 referencias
    public Tarea Tarea { get; set; }
    0 referencias
    public string Descripcion { get; set; }
    0 referencias
    public bool Realizado { get; set; }
    0 referencias
    public int Orden { get; set; }
}
```

```
1 reference
public class IndiceCuentasViewModel
{
    1 reference
    public string TipoCuenta { get; set; }
    2 references
    public IEnumerable<Cuenta> Cuentas { get; set; }
    0 referencias
    public decimal Balance => Cuentas.Sum(x => x.Balance);
}
```

BBDD nueva

Creamos una clase por cada tabla, a dichas clases se las suele llamar entidades



En dicha clase Tarea, implementamos como propiedades todos los campos que vaya a llevar la tabla.

Para evitar warning en los campos String o bien ? o bien doble clic en el nombre del proyecto

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net6.0</TargetFramework>
    <Nullable>disable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Entity
    <PackageReference Include="Microsoft.Entity
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; c
    </PackageReference>
  </ItemGroup>
</Project>
```

Una vez implementada la clase, para que se convierta en una “entidad” debemos configurarla en la clase que creamos para el contexto

```
Public DBSet<Clase> nombre propiedad{get;set;}
```

```
Public DBSet<Tarea> Tareas{get;set;}
```

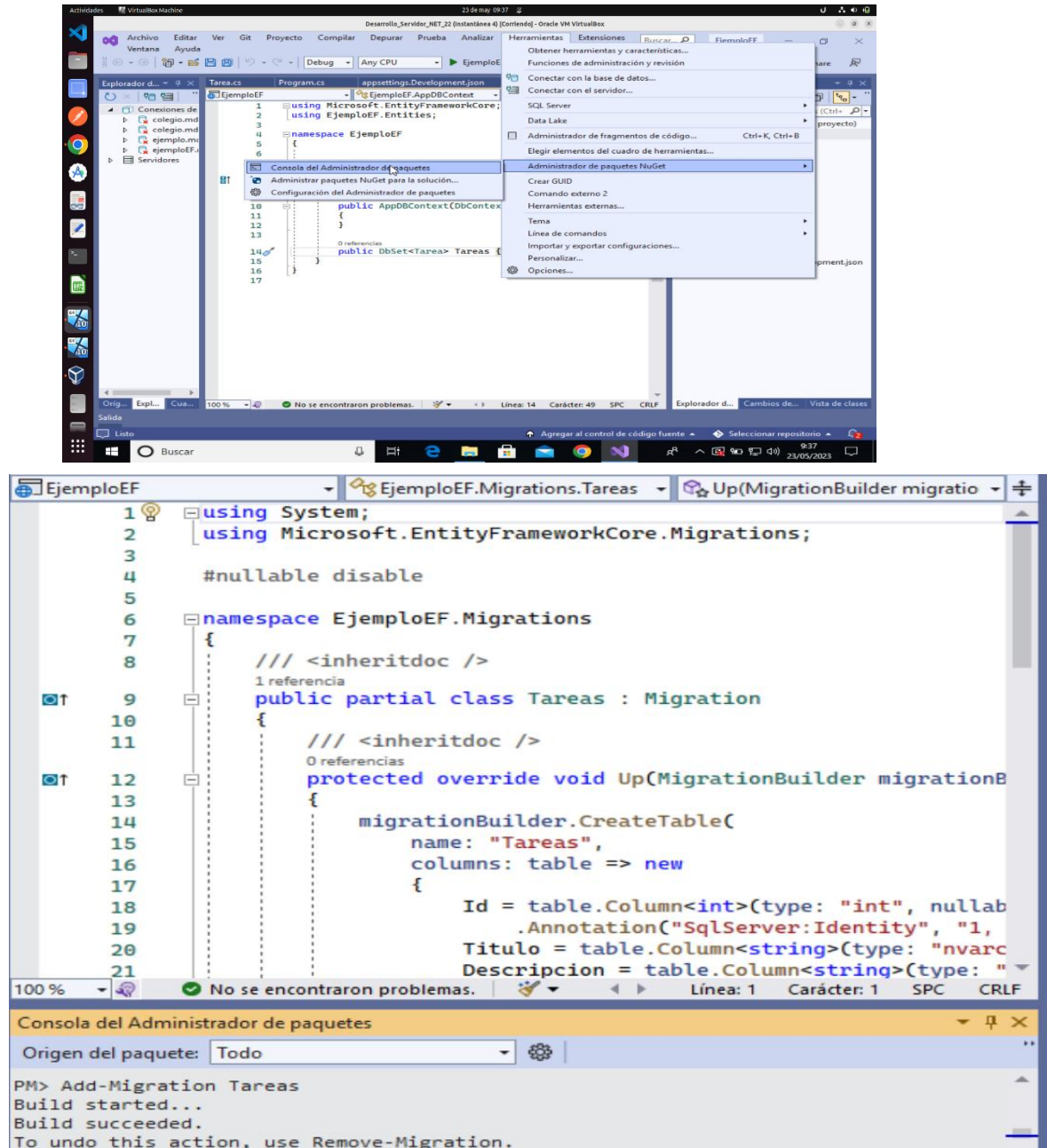
Esto creará una tabla Tareas. Para crear tanto la tabla tenemos que “migrar”

Migraciones:

Representación en c# de los cambios que ocurren en la bbdd. Para hacer una migración usamos Manage Package Console

a) **Crear la migración:** Add-Migration nombre

Se crea una clase en las migraciones



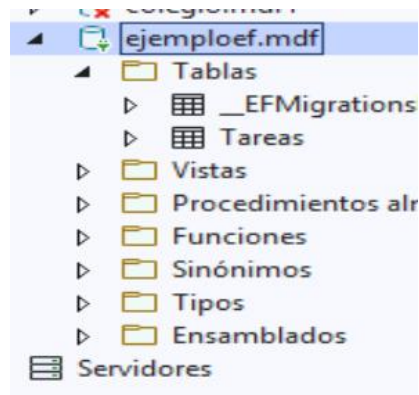
Cuando ejecutemos las migraciones esa función Up se ejecutará y vemos que incluye la creación de campos con nuestras propiedades. Podemos modificar esos campos para que las columnas de la bbdd se ajusten a lo que queremos, primarykey, null, tamaño...

En esta clase también tenemos la función Down por si queremos eliminar la migración con el comando Remove-Migration

b) **Ejecutar la migración:** Update-Database. El archivo mdf tiene que existir

```
PM> Update-Database
Build started...
Build succeeded.
Microsoft.EntityFrameworkCore.Database.Command[20101]
    Executed DbCommand (20ms) [Parameters=[], CommandType='Text',
CommandTimeout='30']
    SELECT 1
Microsoft.EntityFrameworkCore.Database.Command[20101]
    Executed DbCommand (20ms) [Parameters=[], CommandType='Text',
CommandTimeout='30']
```

Podemos ver que nos crea la tabla Tareas con los campos tal y como están en la función Up y otra tabla con datos de las migraciones que vayamos haciendo



Algunas convenciones a tener en cuenta:

1. Si una propiedad de la clase se llama Id, en la tabla será clave primaria
2. Los campos string los genera con longitud máxima de varchar y con posibilidad de ser nulos
3. Los campos int, DateTime los genera no nulos

Evidentemente podemos configurar de otra manera: anotaciones de datos y API

Anotaciones de datos

Conjunto de atributos que podemos usar para configurar las columnas. Ya las vimos en las validaciones. Si añadimos algo creamos una nueva migración

```
Done.
PM> Add-Migration TituloConfigurado
Build started...
Build succeeded.
An operation was scaffolded that may result in the loss of data. Please review the
migration for accuracy.
To undo this action, use Remove-Migration.
PM>
```

```
1 referencia 1 referencia
public partial class TituloConfigurado : Migration
{
    /// <inheritdoc />
    0 referencias
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AlterColumn<string>(
            [?] (parámetro)(MigrationBuilder migrationBuilder)
            The MigrationBuilder that will build the operations.
            maxLength: 200,
            nullable: false,
            defaultValue: "",
            oldClrType: typeof(string),
            oldType: "nvarchar(max)",
            oldNullable: true);
    }
}
```

Podemos observar que la función Up ha cambiado con respecto a la de la migración anterior

Para que esta modificación tenga efecto hay que ejecutar la migración

API

Conjunto de métodos que podemos usar para configurar. Es la herramienta más poderosa del EF. Todo aquello que no podamos hacer directamente (por convención) o a través de las anotaciones de datos, debemos hacerlo con el API en la clase contexto.

En concreto debemos sobrescribir el método `onModelCreating`. A través de su parámetro `modelbuilder` podemos configurar tanto las entidades como sus propiedades.

```
0 referencias
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<Tarea>().Property(d => d.Descripcion).HasMaxLength(300).IsRequired();
}
```

La ventaja que tienen las anotaciones de datos es que las usamos también para validar.

Ejemplo: Añadimos otra entidad `Item` (los sucesivos ítems que compondrán cada tarea de nuestra bbdd). Relacionamos ambas entidades

Conceptos que destacar del ejemplo:

- Propiedades que son claves foráneas: para que se configuren las relaciones en la bbdd
- Propiedades de navegación: para permitirnos ejecutar consultas sin necesidad de hacer inner join. Utilizando funciones sencillas. Para configurar el tipo de relación (uno a muchos, muchos a muchos...)
- Tipo Guid